

REPUBLIC OF THE PHILIPPINES

Polytechnic University of the Philippines

College of Computer and Information Sciences

Sta. Mesa, Manila

LAlgo

Design and Validation of a Unified DFA Lexer

Table-driven lexical analysis in C99

GROUP 4 | BSCS 3-1

Buenaventura, Harvey L.

De Mesa, Joshua C.

Dela Rosa, Mico Von Adriel A.

Magalona, Adriel M.

Morales, Timothy Gabriel

Nevado, Sean Andrew M.

Salvacion, Dean James G.

Submitted to

Prof. Ria A. Sagum

Principles of Programming Languages (COSC 303)

Implementation snapshot | 30 October 2026

Abstract

LAlgo is a proposed programming language for expressing algorithm-oriented source programs. This paper documents its implemented lexical front end: a C99 scanner that recognizes tokens using one deterministic finite automaton (DFA), one start state, and one transition table. The selected implementation constructs 254 explicit states and 83 input classes. Reserved spellings are embedded in the automaton through paired keyword-prefix paths, allowing uppercase recognition and capitalization diagnostics without comparing completed lexemes against a keyword list. A longest-match scanner adds source positions, newline metadata, integer-range checks, and recovery for malformed input. Comments and invalid lexemes remain visible in a token listing written to `Symbol Table.txt`. A reproducible suite passes 31 of 31 local black-box cases, including the supplied positive and negative examples. These results establish behavior for the tested lexical contracts; parsing, semantic analysis, algorithm execution, and instructor acceptance are outside this study.

Keywords: lexical analysis; deterministic finite automaton; maximal munch; keyword trie; error recovery; C99.

1 Introduction and Implementation Scope

A lexical analyzer converts source bytes into a stream of recognizable units. For LAlgo, these units include names, constants, uppercase reserved words, operators, punctuation, comments, and end-of-file. The practical requirement is to make this recognition inspectable: each token can be traced to a path from the same start state and to its original source location.

The earlier language proposal and subsequent lexical specification establish the project context [2, 3]. This paper concentrates on the actual selected program, `lexical_analyzer/lalgo_unified.c` [1]. The older `lalgo_lexer.c` is retained as a historical implementation and is not the implementation evaluated here.

1.1 Objectives

The implemented front end aims to recognize the specified token vocabulary, distinguish keywords from identifiers using table transitions, preserve useful source metadata, and report lexical errors while continuing where recovery is defined. Its trace mode exposes state changes for explanation and inspection.

Scope boundary. Recognition of `WHILE`, `ARRAY`, or `CARET` does not execute a loop, construct an array, or evaluate an exponent. The sample programs demonstrate tokenization rather than executable language semantics.

2 Source and Token Contracts

2.1 Input and output

The command line accepts a plain-text source file whose extension is `.lal`; the extension check is case-insensitive. Embedded NUL bytes are rejected before scanning. CRLF pairs are normalized to LF. An isolated carriage return remains an invalid source byte. Space, tab, and LF are whitespace. Locations begin at line 1, column 1; a tab advances one column rather than expanding to a display tab stop.

Each token stores its type, original lexeme, starting line and column, `precededByNewline`, and `suspectedKeyword`. The program prints a token table and diagnostics, then writes `Symbol Table.txt` in the current working directory. That file lists token occurrences, including repeated identifiers, comments, and invalid lexemes; EOF is omitted from it.

The output filename uses "Symbol Table," but the implemented data is a lexical occurrence listing. It does not resolve declarations, scopes, types, identifier uniqueness, or storage addresses.

2.2 Reserved spellings

Group	Uppercase spellings
Data types	`INTEGER DECIMAL CHARACTER BOOLEAN ARRAY STRING`
Control flow	`IF ELSE WHILE FOR`
Input/output	`INPUT OUTPUT`
Boolean constants	`TRUE FALSE`
Word operators	`DIV AND OR NOT`
Future-reserved	`ALGORITHM RETURN TRACE INVARIANT`
Noise words	`SET THE`

The first five rows contain 18 current reserved spellings. Four future-reserved words and two noise words bring the total to 24. The lexer emits noise-word tokens; any later decision to omit them from an abstract syntax tree belongs to a parser. Future-reserved tokens likewise carry no implemented runtime behavior.

2.3 Identifiers and capitalization

Identifiers follow `[A-Za-z][A-Za-z0-9_]*`. An initial underscore is invalid. Uppercase whole-word matches receive reserved token types. `while` and `While` remain identifiers with `suspectedKeyword=true`. `while1`, `WHILE_`, and the incomplete prefix `WHIL` are ordinary identifiers without that flag. The diagnostic does not silently convert a name into a keyword.

3 One Unified Deterministic Automaton

3.1 Representation

Let Q_e be the 254 explicit implementation states and let Σ_c be the 83 input classes, including virtual EOF. Recognition begins at q_0. The program stores δ as `DELTA[state][class]` and records acceptance in `ACC_TOK`. Missing transitions use -1 and stop lookahead. For a total mathematical DFA, add one implicit non-accepting sink $q_\perp$:

$$M = (Q_e \cup \{q_\perp\}, \Sigma_c, \delta, q_0, F), \qquad \delta(q_\perp, c) = q_\perp.$$

Thus 254 is the stored-state count, not the size of the totalized state set. Acceptance actions distinguish token production, whitespace skipping, and non-acceptance. The scanner emits EOF directly when no source remains.

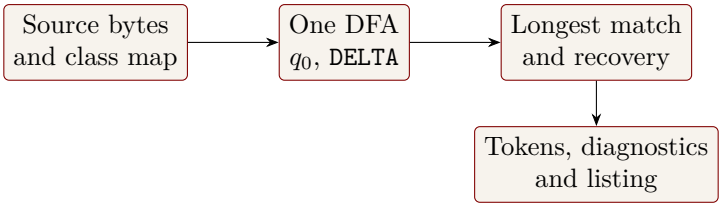


Figure 1. Processing architecture. The boxes show program stages, not independent token-recognition automata.

3.2 Classes and integrated keyword paths

Each uppercase and lowercase letter has its own class, giving 52 letter classes. The remaining 31 classes cover digits, relevant punctuation and whitespace, other printable ASCII, illegal bytes, and EOF. A 256-entry map classifies input bytes before transition lookup.

At startup, `build_trie` constructs shared prefixes from the 24 reserved spellings. Every non-root trie node becomes two DFA states: an exact-uppercase state `E:prefix` and a folded-case state `F:prefix`. An exact terminal accepts the corresponding reserved token. A folded terminal accepts an identifier with the suspicion flag. Digits, underscores, or diverging letters lead to a general identifier state, which continues the full name.

The keyword array is construction data. During scanning, keyword recognition uses the same transition lookup as every other token family; it performs no completed-lexeme string comparison. The generated full transition appendix in the companion specification can be recreated using `lexical_analyzer/export_unified_dfa.py` [4].

4 Numbers, Literals, Operators, and Comments

4.1 Numbers and punctuation boundaries

An integer is one or more digits; a decimal requires digits on both sides of one dot. Leading zeros are accepted. Signs are separate tokens: `-2` becomes `MINUS INTEGER`. Integer magnitudes above 2,147,483,647 generate an `INVALID` token and a diagnostic. This range check runs after the DFA recognizes the digit sequence.

Input	Observed lexical result
`3.14`	One `DECIMAL` token.
`3.`	`INTEGER(3)`, then `DOT(.)`; lookahead backs up.
`.5`	`DOT(.)`, then `INTEGER(5)`.
`2low`	One malformed-number `INVALID` token.
`1.5abc`	One malformed-number `INVALID` token.
`1..2`	`INTEGER DOT DOT INTEGER`.

The lexer preserves decimal text; it does not calculate or store a floating-point numeric value. Numeric interpretation, expression precedence, and associativity are later language-processing concerns.

4.2 Quoted literals

Strings use double quotes and may be empty. Character literals use single quotes and require exactly one permitted body character. Both accept permitted printable ASCII and tabs, while rejecting backslashes, LF, and illegal bytes. There is no escape syntax. A quote matching the opening delimiter closes the literal; the other quote character may appear in its body.

4.3 Operators and delimiters

Symbolic arithmetic tokens are `+ - * / % ^`; `DIV` is the seventh arithmetic operation's word token. Assignment is `=`; relational tokens are `== != < <= > >=`. A lone `!` is invalid. Parentheses, square brackets, braces, comma, semicolon, colon, and dot have individual accepting states. A two-character operator supersedes its shorter accepted prefix under the longest-match rule.

4.4 Comment boundaries

`//` consumes through the end of its line without consuming LF. `/*` consumes through the first `*/`; block comments do not nest. Comment bodies can traverse otherwise illegal bytes so the scanner can locate the boundary, then report those bytes at their locations. Comments are retained as auxiliary tokens. An unclosed block comment consumes the remaining input as one invalid lexeme and produces an unterminated-comment diagnostic.

5 Scanning, Metadata, and Recovery

5.1 Longest accepted prefix

For each candidate, `dfa_run` starts at q_0, advances through defined transitions, and remembers the most recent accepting state and input length. When a transition is missing, the scanner selects that remembered prefix. The following pseudocode summarizes ordinary matching; committed errors are handled before fallback.

```
state = q0; offset = 0; last_accept = NONE
while DELTA[state][class_at(start + offset)] is defined:
    state = DELTA[state][class_at(start + offset)]
    offset = offset + 1
    if state has an accepting action:
        last_accept = (state, offset)
if stop_state is committed:
    recover malformed literal or block comment
else if last_accept exists:
    emit token, skip whitespace, or check numeric range
else:
    report and consume one illegal byte
restart at q0 for the next candidate
```

5.2 Why committed states matter

The slash in `/* open` is initially a valid division token. Once the scanner enters the block-comment body, falling back to that slash would hide the missing terminator. `COMMIT` marks open strings, character literals, and block comments so an unfinished construct takes priority over an earlier acceptance. Quoted-literal recovery consumes through a matching closing quote, or stops before LF or EOF. Block-comment recovery consumes to EOF.

Malformed digit/name adjacency follows a separate accepting `BAD_NUM` path. Range failures convert a recognized integer candidate into `INVALID`. Standalone illegal bytes consume one byte, ensuring progress.

5.3 Newline metadata and diagnostics

`advance` updates line and column positions for consumed input. Two pending-newline flags distinguish normal tokens from auxiliary comment and invalid tokens. An auxiliary row can display its own newline relationship without erasing the flag needed by the next normal token. LF inside a consumed block comment also affects the next token's metadata. These flags preserve information; they do not implement statement boundaries.

Diagnostics include the offending construct's start position, or the specific illegal-byte position within a comment. Exit status is 0 for a scan with no lexical errors, 1 for a scan reporting lexical errors, and 2 for usage or file handling failures. The command overwrites the listing on each completed scan.

6 Worked Tokenization Example

The following compact source illustrates uppercase recognition, a lowercase keyword warning, decimal-dot backtracking, and newline propagation through a comment. The displayed positions refer to normalized source text.

```
INTEGER low = 3.
// boundary
while low <= 5
```

Position	Token type	Lexeme	Newline	Suspected
1:1	KW_INTEGER	INTEGER	false	false
1:9	IDENT	low	false	false
1:13	ASSIGN	=	false	false
1:15	INTEGER	3	false	false
1:16	DOT	.	false	false
2:1	LINE_COMMENT	// boundary	true	false
3:1	IDENT	while	true	true
3:7	IDENT	low	false	false
3:11	LE	<=	false	false
3:14	INTEGER	5	false	false
4:1	EOF	(empty)	true	false

Table 1. Expected stream for the example with a final LF. EOF appears on screen and is excluded from the file listing.

6.1 Interpreting the paths

For `INTEGER`, the automaton follows exact-uppercase prefix states and ends at the reserved data-type token. For `while`, its lowercase first letter selects a folded-case path; the completed spelling accepts `IDENT` with the suspicion flag. If a digit were appended, the word would continue into the general identifier state and lose that flag.

For `3.`, the digit state accepts an integer, then the dot reaches a non-accepting decimal-prefix state. LF cannot complete the decimal, so the scanner emits the earlier integer and restarts on the dot. For `<=`, the accepting less-than state extends to the accepting less-than-or-equal state.

6.2 Connection to the supplied program

`lexical_analyzer/sample.lal` contains the illustrative binary-search source. The current implementation produces 99 file-listing entries, including one line comment, plus EOF on screen. Its zero lexical diagnostics demonstrate successful recognition of the source. They do not establish that the program parses, that its search terminates, or that LAlgo can execute it.

7 Reproducible Local Validation

7.1 Method and results

The test runner [5] invokes the compiled executable in an isolated temporary directory for each case. It checks the exit status, diagnostic count, EOF presence, and listing contract. Cases add explicit token expectations or metadata assertions where relevant. Expected token sequences are authored independently of the DFA tables; the runner does not call internal recognition functions. The source hash is recorded in `build/tests/results.json`.

Measured result: 31 / 31 local cases pass for the selected source snapshot. The build uses `-std=c99 -Wall -Wextra -Wpedantic -Werror`. No performance benchmark or exhaustive input-space proof is claimed.

Contract area	Checks included in the suite
Reserved words	All 24 spellings, folded-case diagnostics, partial prefixes, and suffixes that turn keywords into identifiers.
Names and numbers	Identifier alphabet, illegal initial underscore, leading zeros, maximum integer, overflow, decimals, dot fallback, and number/name adjacency.
Punctuation	Arithmetic, assignment, relational operators, brackets, delimiters, and lone-exclamation recovery.
Literals and comments	Empty strings, tabs, exact character length, forbidden backslashes, unterminated constructs, non-nesting, and illegal comment bytes.
Positions and files	Newline across comments, CRLF normalization, tab columns, isolated CR, empty input, file extension, NUL rejection, and trace dimensions.
Supplied examples	Positive binary-search sample, supplementary demo, and negative example with seven diagnostics and seven invalid entries.

7.2 Evidence boundaries

The tests verify selected observable behaviors, rather than every transition or every possible malformed input. Exporting 254 states and 83 classes confirms the implementation's current dimensions; it does not prove minimality of the automaton. Token counts are specific to the supplied files and change when their text changes.

The implementation retains the source, lexemes, and diagnostics in memory; this is not a streaming interface. Very large files, allocation failure, platform-specific integer and file-size limits, and timing behavior are not covered by these checks. The report records local validation, not instructor review, course grading, or external platform acceptance.

8 Reproduction, Limitations, and Conclusion

8.1 Build and run

From the repository root, run:

```
make test
./build/lalgo lexical_analyzer/sample.lal
./build/lalgo lexical_analyzer/demo.lal t
./publish.sh --build-only
```

The first command compiles the selected C99 lexer and runs the contract suite. The next two demonstrate ordinary output and optional DFA tracing. The last command repeats validation, regenerates the full DFA appendix, and builds both the language specification and this paper using Python 3 and a LaTeX installation with `latexmk`. It does not publish to the papers website.

8.2 Remaining work

The lexer does not implement grammar recognition, type checking, scope resolution, statement-boundary rules, precedence evaluation, or runtime execution. Unicode identifiers, numeric exponent notation, escape sequences, and nested comments are unsupported. Listing output is intended for inspection: auxiliary lexemes are escaped and display-limited, and regular long lexemes can widen the console layout. The listing is not a lossless serialization format.

Further work can define a parser contract around the existing token metadata, specify semantics for current and future-reserved vocabulary, and broaden tests with generated inputs. Those steps should preserve the distinction between recognizing a source form and implementing its meaning.

8.3 Conclusion

LAlgo's current lexical front end combines token families and keyword prefixes in one table-driven DFA. Longest-match selection handles shared prefixes, while committed-state recovery prevents malformed quoted constructs and block comments from disappearing into shorter tokens. Source positions, capitalization flags, newline metadata, and auxiliary rows make its output inspectable. The passing local suite provides a reproducible baseline for subsequent compiler work.

Project Sources

[1] Group 4. *Unified LAlgo lexical analyzer*. Repository source: `lexical_analyzer/lalgo_unified.c`. Snapshot evaluated 30 October 2026.

[2] Group 4. *LAlgo Language Design Specification*. `lalgo_language_specification.tex`; companion PDF in this repository.

[3] Group 4. *Principles of Programming Language Proposal*. Original proposal PDF retained in the repository.

[4] Group 4. *Unified DFA export utility*. `lexical_analyzer/export_unified_dfa.py`.

[5] Group 4 project repository. *Lexer contract validation*. `tests/test_lexer.py`; reproducible report in `docs/VALIDATION.md`.